



中国海洋大学 海德学院
HAIDE COLLEGE, OCEAN UNIVERSITY OF CHINA

PRS-Net 论文复现：实现逻辑与技术调整

孙景昱

中国海洋大学, 海德学院
Ocean University of China, Haide College

2026 年 5 月 25 日

- 1 数学解释
- 2 技术实现
- 3 验证过程
- 4 复现流程与代码结构
- 5 实际复现中的调整
- 6 实验结果与分析
- 7 总结

PRS-Net 是一种用于检测 3D 模型的对称性的无监督学习算法, 其核心算法采用了卷积神经网络 (CNN), 损失函数的构造则综合采用了对称距离误差 (Symmetry Distance Error, SDE), 同时采用对称元素 (对称平面或对称轴) 所表示的矩阵范数作为惩罚项来减少重复对称元素的出现.

这种构造损失函数的方法通过找到对称点后计算 SDE 距离实现, 其中大部分计算均可通过 PyTorch 实现.

镜面对称的表示

本文主要讨论了两种对称情形：镜面对称 (Reflection) 和旋转对称 (Rotation)，根据解析几何的知识，镜面对称的两点应当满足下述条件：

$$q'_i = q_i - 2 \frac{q_i \cdot n + d}{\|n\|^2} \cdot n, \quad (1)$$

其中 q 为未经变换的点， q' 为经过变换后对应的点， n 为平面单位法向量， d 为取定 n 后平面的偏移量。

旋转对称的表示

旋转对称应满足下述条件:

$$q'_j = p \cdot q_j \cdot p^{-1}, \quad (2)$$

其中 q 为未经变换的点, q' 为经过变换后对应的点, p 为旋转操作的四元数. 这种计算方法较矩阵计算更为简洁, 且可以避免万向节锁问题.

为方便处理, 这里额外定义四元数的乘法计算, 同时参与运算的 q 和 q' 也采用四元数形式表示.

总对称损失函数

根据两种不同对称方式得到的点, 计算如下距离作为单个点的误差:

$$D_{(i \text{ or } j)} = \min_{o \in O} \|q_{(i \text{ or } j)} - o\|_2, \quad (3)$$

其中 O 表示某个物体表面的点. 这里采用 L2 范数, 并统一采用距离归一化后的值进行计算, 以保证各个检测物体的损失函数在计算时是等量纲的.

于是总体对称的损失函数可表示为

$$L_{sd} = \sum_{k=1}^3 D_i^k + \sum_{k=1}^3 D_j^k \quad (4)$$

其中 k 表示对称元素的序号.

对于可能重复的对称元素, 将其特征标准化后构成矩阵, 并计算其与单位阵的正交接近程度, 可作为对高度重合的对称元素的惩罚项, 即在一定程度上避免了重复对称元素的得到:

$$L_r = \|M'_i\|_F^2 + \|M'_j\|_F^2, \quad (5)$$

其中 $\|\cdot\|_F$ 表示 Frobenius 范数. $M'_i = M_i M_i^T - I$, $M'_j = M_j M_j^T - I$ 分别表示两种对称元素与单位阵的正交接近程度.

于是总损失函数定义为

$$L = L_{sd} + w_r \cdot L_r, \quad (6)$$

其中 w_r 表示惩罚项权重参数.

卷积神经网络 (CNN)

本文所用算法是无监督学习, 这主要是因为大多数数据通常本身不包含对称信息, 即只需要利用数据本身包含点的对称信息来计算, 极大提高了算法的普适性.

这里 CNN 输入数据大小为 $x = (B, 1, 32, 32, 32)$, 表示 B 个物体的体素 (Voxel), 以及空间内是否为实心, 方便后续选取表面点计算单点 SDE.

借助五层卷积层和扁平层, CNN 将输入数据 x 转化为 64 维的向量. 随后利用全连接层分别输出对称平面和对称轴的表示形式.

本次复现采用的是 ModelNet40 数据, 以对比其与 ShapeNet 数据的表现. 因此需要构建一个前处理文件以对 *.off 文件进行处理, 获取表面点的信息并将其转换为 CNN 可读的 *.pt 数据:

$$x_{grid} = s * (x - x_{center}) + c \quad (7)$$

其中 s 为放缩比例, 确保整个物体都落在 32^3 内, x_{center} 为全部数据点的中心, c 为 32^3 的中心.

记上述调整完成后的全体数据为 X , 则 $X \subset 32^3$, 于是有

$$X' = X \cdot R \quad (8)$$

其中 R 为随机旋转矩阵, 以增强学习效果.

同时为方便后续计算点与物体表面的 SDE, 构造

$$Q = \{q_k, k = 1, \dots, 1000\} \quad (9)$$

表示物体表面.

最短距离的预计算

为减少实际训练时间, 这里预计算物体表面到各个点的最小距离, 记作

$$C(g) = \operatorname{argmin}_{o \in O} \|g - o\|_2 \quad (10)$$

其中 g 为 Voxel 坐标.

最终的 *.pt 文件中存有 Voxels, 表面点坐标和最近的整数坐标点等信息, 以确保后续读取不存在其他问题.

候选对称元素的验证

网络输出的结果并不直接作为最终对称元素, 而是先作为候选结果进行验证. 对于每一个候选对称平面或旋转轴, 将物体表面点进行对应的对称变换, 再计算变换后点集与原物体之间的 SDE:

$$E(S, \tau) = \frac{1}{|S|} \sum_{p \in S} \text{dist}(\tau(p), S)^2. \quad (11)$$

若误差满足

$$E(S, \tau) \leq 4 \times 10^{-4}, \quad (12)$$

则认为该候选对称元素通过验证.

对称平面的验证

对于网络输出的候选平面

$$P_i = (n_i, d_i), \quad (13)$$

首先计算每个点关于该平面的镜像点:

$$p' = p - 2 \frac{p \cdot n_i + d_i}{\|n_i\|^2} n_i. \quad (14)$$

随后计算镜像后点集与原物体之间的 SDE. 若该误差低于阈值, 则认为该平面是一个有效的镜面对称平面.

对称轴的验证

对于网络输出的四元数

$$q = (w, x, y, z), \quad (15)$$

取其虚部方向作为旋转轴:

$$a = \frac{(x, y, z)}{\|(x, y, z)\|}. \quad (16)$$

随后依次检查该轴在不同旋转角度下的 SDE:

$$\theta = 1^\circ, 2^\circ, \dots, 359^\circ. \quad (17)$$

只有当所有角度下的误差均低于阈值时, 才认为该轴是有效旋转对称轴.

本次复现的整体流程如下:

OFF 模型 $\rightarrow$ 预处理 $\rightarrow$ CNN 预测 $\rightarrow$ 损失函数优化 $\rightarrow$ 验证与可视化

其中预处理阶段负责生成网络输入和距离查询表, 训练阶段负责无监督优化对称参数, 验证阶段则根据 SDE 阈值判断候选对称元素是否有效.

复现代码结构如下:

文件	功能
preprocess.py	将 *.off 模型转换为 *.pt 训练数据
dataset.py	读取预处理后的训练样本
model.py	定义 PRS-Net 的 CNN 网络结构
loss.py	定义对称距离损失和正则项
train.py	执行模型训练并保存参数
validation.py	根据 SDE 验证对称平面和旋转轴
visualize.py	输出候选参数并生成交互式可视化结果

验证阶段的问题

初始实现中, 两种对称的验证均使用

$$S_{\text{voxel}} = \{g = (x, y, z) \mid V(g) = 1\} \quad (18)$$

作为点集, 但 Rotation 验证需要检查绕某一轴旋转 1° 到 359° 后的误差. 对于离散 Voxel 点集而言, 连续旋转后的点很难落在原有 Voxel 点集附近, 因此会人为放大 Rotation 的 SDE.

这直接导致了有相当部分满足旋转对称的物体无法识别出有效的对称轴, 因此对原有逻辑进行了优化.

为了分别适配两类对称元素, 本次复现最终采用了分离的验证点集:

$$\begin{cases} \text{Reflection} : S_{\text{voxel}}, \\ \text{Rotation} : Q = \{q_k\}_{k=1}^{1000}. \end{cases} \quad (19)$$

即镜面对称继续使用 Voxel 占据点进行验证, 而旋转对称使用预处理阶段采样得到的 1000 个表面点进行验证.

这样可以保留 Reflection 验证的稳定性, 同时减少 Rotation 连续角度验证中的离散化误差.

对于网络输出的四元数 $p = (w, x, y, z)$, 取其虚部方向作为旋转轴:

$$a = \frac{(x, y, z)}{\|(x, y, z)\|}. \quad (20)$$

随后验证该轴在 $\theta \in \{1^\circ, 2^\circ, \dots, 359^\circ\}$ 下是否保持对称, 若所有角度下的 SDE 均满足

$$E(S, \theta) \leq 4 \times 10^{-4}, \quad (21)$$

则认为该轴为有效旋转对称轴.

不同验证方式的对比

这里对 vase, bottle, cup, lamp, flower_pot, plant, stool 七种类别的物体数据进行批量验证, 得到如下结果:

验证方式	有效旋转轴样本数	有效平面总数
全部使用 S_{voxel}	0 / 140	129
全部使用 Q	31 / 140	15
分离验证方式	31 / 140	131

可以看到, 采用分离验证方式能够在保留对称平面结果的同时, 显著改善对称轴的识别.

旋转类别统计

使用分离验证方式后, 各类别有效旋转轴结果如下:

类别	有效旋转轴样本数
bottle	15 / 20
vase	8 / 20
lamp	6 / 20
flower_pot	2 / 20
cup	0 / 20
plant	0 / 20
stool	0 / 20

其中 bottle 和 vase 数据的旋转轴主要集中在 z 方向, 与物体本身的几何结构基本一致.

当前复现结果与论文仍存在差距, 主要原因包括:

- ModelNet40 与论文使用的 ShapeNet 数据分布不同;
- ModelNet40 中并非所有模型都具有严格对称性, 但训练时并未严格挑选;
- 训练阶段只优化网络输出四元数对应的旋转, 而验证阶段要求整根轴在 1° 到 359° 均成立;
- Voxel 分辨率为 32^3 , 对连续旋转对称存在一定离散化误差.

本次复现基本跑通了 PRS-Net 从数据处理到结果验证的完整流程:

- 将 ModelNet40 中的 off 模型整理为网络可直接读取的训练数据;
- 搭建了以 3D CNN 为主体的参数预测网络;
- 按照论文思路实现了镜面对称和旋转对称的无监督损失;
- 实现了候选对称平面、旋转轴的误差计算与可视化输出;
- 针对 Rotation 结果长期不通过的问题, 对验证阶段的点集选取进行了调整.

后续改进方向

从目前结果来看, 后续更值得继续推进的方向主要有:

- 换用论文中使用的 ShapeNet 数据, 减少数据集差异带来的影响;
- 对测试样本进行人工筛选, 避免将本身不具备明显对称性的模型纳入统计;
- 重新设计 Rotation 部分的训练约束, 使其更接近验证阶段的连续旋转轴判据;
- 尝试更高分辨率的 Voxel 表示, 减少离散化对 SDE 计算的影响;
- 对 bottle, vase 等具有明显旋转对称性的类别单独训练和分析.

一些问题

1. 两种对称元素完全分开预测会不会效果更好, 即让 CNN 分别学习两种对称的特征, 加快损失函数收敛的速度.
2. 采用 ModelNet40 的训练数据表明, 部分轴对称的结果的 SDE 往往能够控制在和 4×10^{-4} 同一数量级附近, 但被严格剔除, 这部分的阈值是否可以通过有监督学习的方式得到进一步优化.
3. 这个模型能否用除了 PyTorch 以外的其他方式来加速, 现在只用了 1000 个构造样本, 数量也没有达到论文里的实际数据量级.

敬请批评指正!